```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub
import warnings

from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.linear_model import Ridge, Lasso
from sklearn.svm import SVR
from xgboost import XGBRegressor
from sklearn.utils import resample

import scipy.stats as stats
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.metrics import mean_absolute_percentage_error, median_absolute_error
from sklearn.model_selection import cross_val_score, learning_curve
import pickle
```

```
pd.options.display.float_format = '{:.2f}'.format
warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings('ignore')


df = pd.read_csv('/content/website_wata.csv')
```

```
df.describe()
```

	Page Views	Session Duration	Bounce Rate	Time on Page	Previous Visits	Conversion Rate
count	2000.00	2000.00	2000.00	2000.00	2000.00	2000.00
mean	4.95	3.02	0.28	4.03	1.98	0.98
std	2.18	3.10	0.16	2.89	1.43	0.07
min	0.00	0.00	0.01	0.07	0.00	0.34
25%	3.00	0.82	0.16	1.94	1.00	1.00
50%	5.00	1.99	0.27	3.32	2.00	1.00
75%	6.00	4.20	0.39	5.41	3.00	1.00
max	14.00	20.29	0.84	24.80	9.00	1.00

```
print(f"Loaded {df.shape[0]} records and {df.shape[1]} columns.")
```

```
Loaded 2000 records and 7 columns.
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 7 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Page Views        2000 non-null   int64
 1   Session Duration  2000 non-null   float64
 2   Bounce Rate       2000 non-null   float64
 3   Traffic Source    2000 non-null   object
 4   Time on Page      2000 non-null   float64
 5   Previous Visits   2000 non-null   int64
 6   Conversion Rate   2000 non-null   float64
dtypes: float64(4), int64(2), object(1)
memory usage: 109.5+ KB
```

```
df.dtypes
```

	0
Page Views	int64
Session Duration	float64
Bounce Rate	float64
Traffic Source	object
Time on Page	float64
Previous Visits	int64
Conversion Rate	float64

dtype: object

```
df.head()
```

	Page Views	Session Duration	Bounce Rate	Traffic Source	Time on Page	Previous Visits	Conversion Rate
0	5	11.05	0.23	Organic	3.89	3	1.00
1	4	3.43	0.39	Social	8.48	0	1.00
2	4	1.62	0.40	Organic	9.64	2	1.00
3	5	3.63	0.18	Organic	2.07	3	1.00
4	5	4.24	0.29	Paid	1.96	5	1.00

Next steps: Generate code with df New interactive sheet

```
df.tail()
```

	Page Views	Session Duration	Bounce Rate	Traffic Source	Time on Page	Previous Visits	Conversion Rate
1995	1	2.72	0.21	Referral	1.32	2	1.00
1996	3	0.39	0.10	Organic	3.82	1	1.00
1997	4	9.90	0.45	Organic	1.29	1	1.00
1998	3	0.39	0.28	Paid	5.04	2	1.00
1999	3	0.88	0.34	Direct	5.19	3	1.00

```
df.columns.tolist()
```

```
['Page Views',
 'Session Duration',
 'Bounce Rate',
 'Traffic Source',
 'Time on Page',
 'Previous Visits',
 'Conversion Rate']
```

```
print(df["Conversion Rate"].describe())
```

```
count   2000.00
mean       0.98
std        0.07
min        0.34
25%        1.00
50%        1.00
75%        1.00
max        1.00
Name: Conversion Rate, dtype: float64
```

```
print(df["Conversion Rate"].describe())
```

```
count   2000.00
mean       0.98
std        0.07
min        0.34
25%        1.00
50%        1.00
75%        1.00
max        1.00
Name: Conversion Rate, dtype: float64
```

Separate majority and minority classes

```
majority = df[df["Conversion Rate"] == 1.0]
minority = df[df["Conversion Rate"] < 1.0]
```

Oversampling the Minority Class

Upsample the minority class to match the size of the majority class

```
minority_upsampled = resample(minority, replace=True, n_samples=len(majority), random_state=42)
```

Combine the majority class with the upsampled minority class

```
df_balanced = pd.concat([majority, minority_upsampled])
```

Check the distribution of classes

```
print(df_balanced["Conversion Rate"].value_counts())
```

```
Conversion Rate
1.00    1773
0.89      17
0.85      16
0.75      16
0.87      16
        ...
0.96       3
0.90       3
1.00       2
0.76       2
0.71       1
Name: count, Length: 228, dtype: int64
```

Undersampling the Majority Class Undersample the majority class to match the size of the minority class

```
majority_sampled = majority.sample(n=len(minority), random_state=42)
```

Combine the undersampled majority class with the minority class

```
df_balanced = pd.concat([majority_sampled, minority])
```

Check the distribution of classes

```
print(df_balanced["Conversion Rate"].value_counts())
```

```
Conversion Rate
1.00    227
0.96      1
0.87      1
0.97      1
0.89      1
       ...
0.79      1
0.57      1
1.00      1
0.94      1
0.95      1
Name: count, Length: 228, dtype: int64
```

```
df.head()
```

	Page Views	Session Duration	Bounce Rate	Traffic Source	Time on Page	Previous Visits	Conversion Rate
0	5	11.05	0.23	Organic	3.89	3	1.00
1	4	3.43	0.39	Social	8.48	0	1.00
2	4	1.62	0.40	Organic	9.64	2	1.00
3	5	3.63	0.18	Organic	2.07	3	1.00
4	5	4.24	0.29	Paid	1.96	5	1.00

Next steps: Generate code with df New interactive sheet

DATA PREPROCESSING

```
df.isnull().sum()
```

	0
Page Views	0
Session Duration	0
Bounce Rate	0
Traffic Source	0
Time on Page	0
Previous Visits	0
Conversion Rate	0

dtype: int64

```
df.isna().sum()
```

	0
Page Views	0
Session Duration	0
Bounce Rate	0
Traffic Source	0
Time on Page	0
Previous Visits	0
Conversion Rate	0

dtype: int64

```
duplicates = df.duplicated()
print(f"Number of duplicate rows: {duplicates.sum()}")
```

```
Number of duplicate rows: 0
```

Suppose df["Conversion Rate"] is in percent (e.g. 34.0 for 34%)

```
df["Conversion Rate"] = df["Conversion Rate"] / 100.0
```

To break the hard ceiling at 1.0, replace exact 1.0's with a tiny epsilon below it:

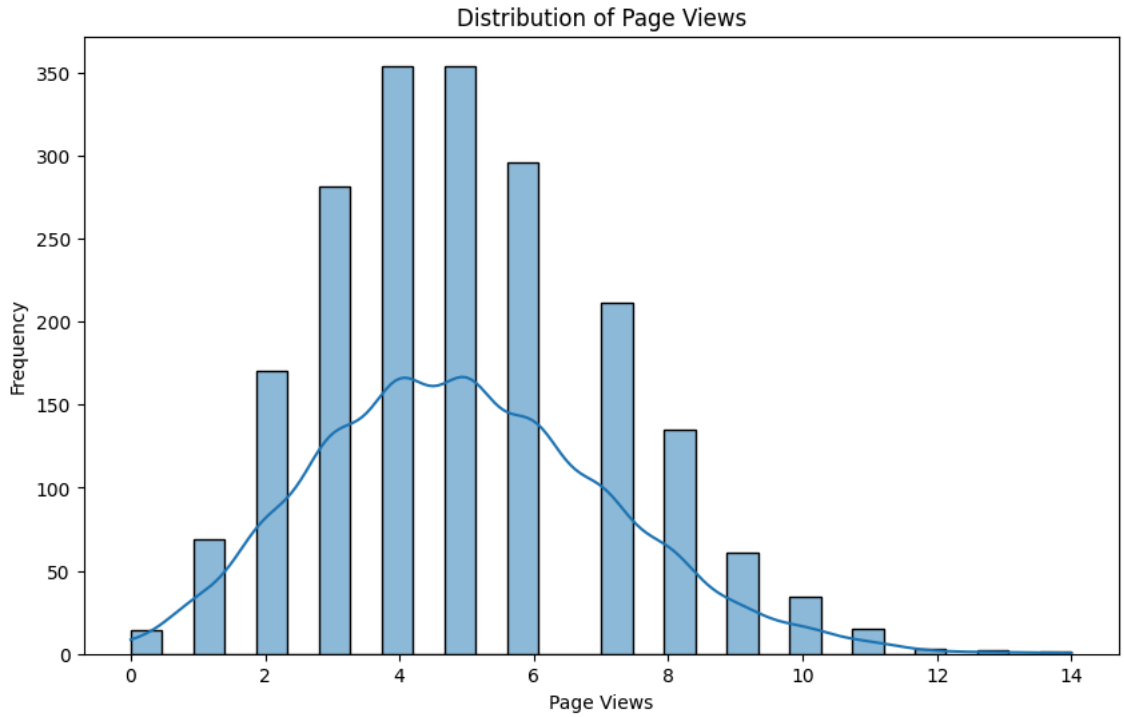
This "jitter" gives the model some variation to learn from at the upper bound

```
eps = 1e-3
df["Conversion Rate"] = df["Conversion Rate"].replace(1.0, 1.0 - eps)
```

VISUALIZATIONS

Distribution of Page Views

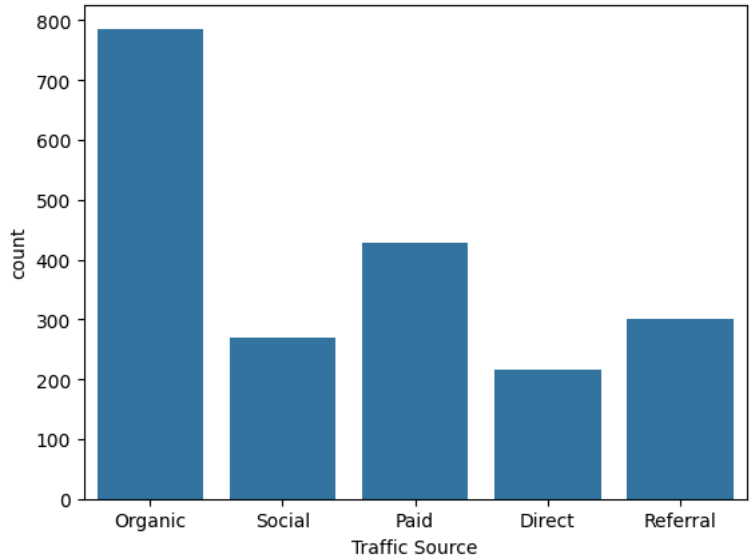
```
plt.figure(figsize=(10, 6))
sns.histplot(df['Page Views'], kde=True)
plt.title('Distribution of Page Views')
plt.xlabel('Page Views')
plt.ylabel('Frequency')
plt.show()
```



Distribution Of Traffic Source

```
sns.countplot(x='Traffic Source',data=df)
```

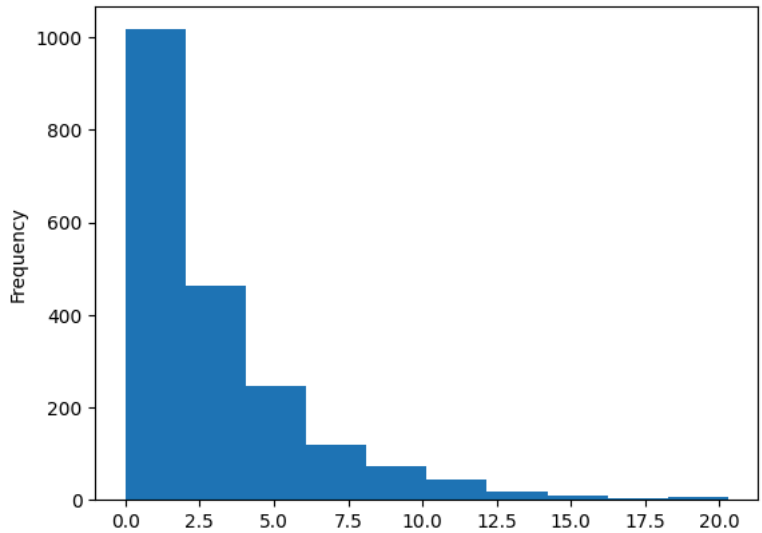
```
<Axes: xlabel='Traffic Source', ylabel='count'>
```



Distribution Of Session Duration

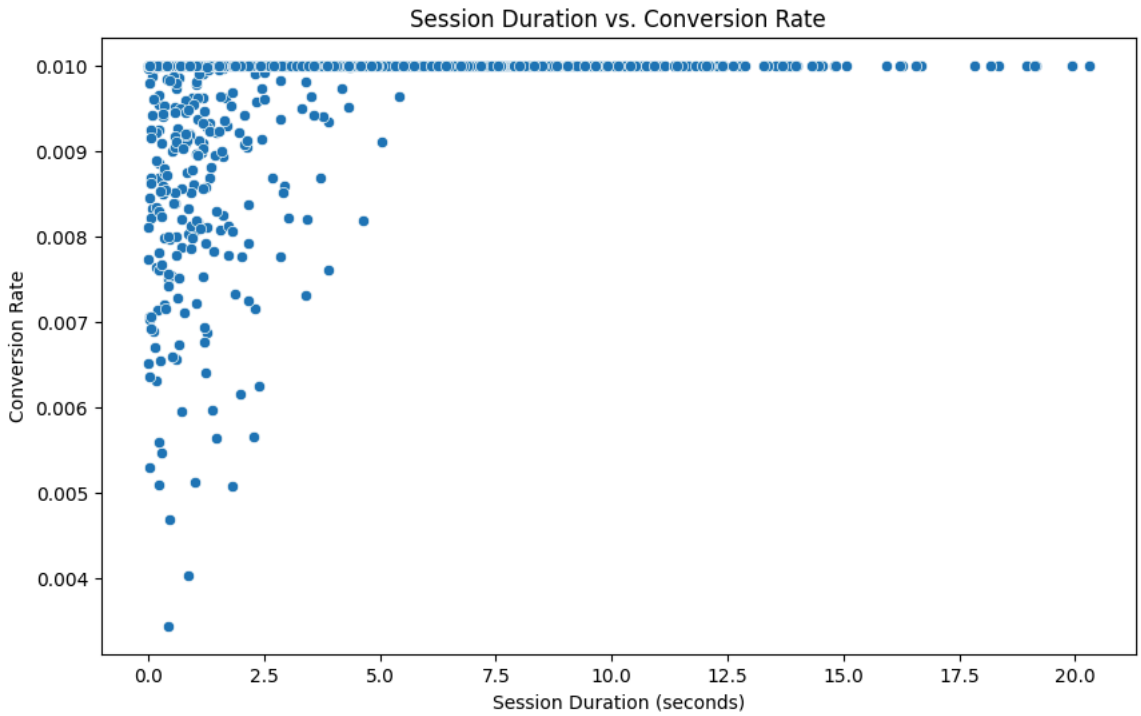
```
df['Session Duration'].plot(kind='hist')
```

```
<Axes: ylabel='Frequency'>
```



Scatter plot of Session Duration vs. Conversion Rate

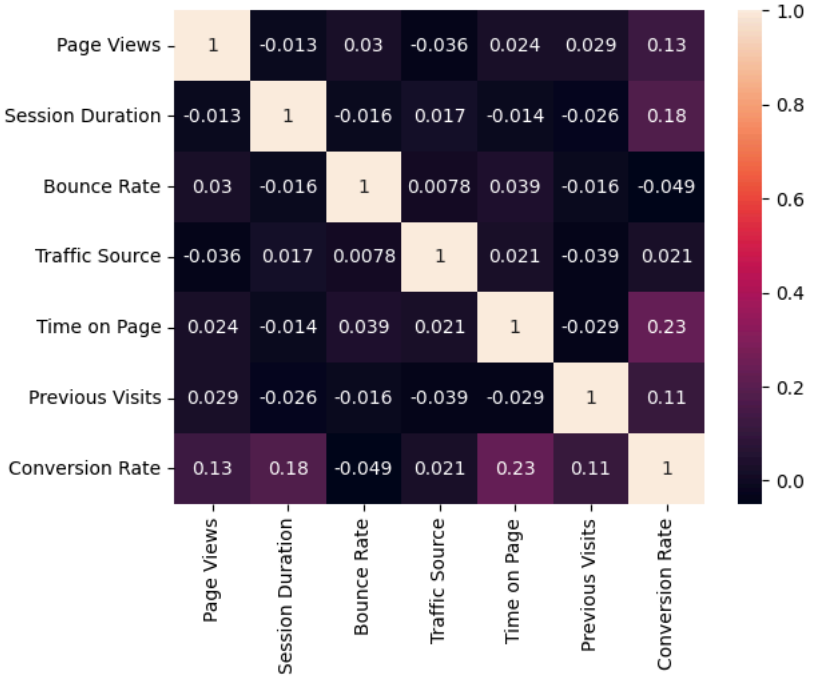
```
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Session Duration', y='Conversion Rate', data=df)
plt.title('Session Duration vs. Conversion Rate')
plt.xlabel('Session Duration (seconds)')
plt.ylabel('Conversion Rate')
plt.show()
```




```
le = LabelEncoder()

df['Traffic Source'] = le.fit_transform(df['Traffic Source'])
sns.heatmap(df.corr(),annot=True)
```

```
<Axes: >
```



CORRELATION HEATMAP

```
numeric_df = df.select_dtypes(include=[np.number])
plt.figure(figsize=(12, 8))
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap')
plt.show()
```

```
df.head()
```

summary of the dataset

```
print(f"Loaded {df.shape[0]} records and {df.shape[1]} columns.")
```

Cleaned data

```
df.to_csv("Cleaned_Data.csv", index=False)
```

```
df
```

MODEL SELECTION

Define Target And Independent Variable

```
X = df.drop("Conversion Rate", axis=1)
y = df["Conversion Rate"]
```

Split data into training and testing sets (80% training, 20% testing)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Initialize models

```
models = {
    'Ridge Regression': Ridge(),
    'Lasso Regression': Lasso(),
    'Random Forest': RandomForestRegressor(),
    'Gradient Boosting': GradientBoostingRegressor(),
    'XGBoost' : XGBRegressor(),
    'Support Vector Regressor': SVR()
}
```

List to store results

```
results = []
```

```
for model_name, model in models.items():

    # Train the model
    model.fit(X_train, y_train)

    # Make predictions
    y_pred = model.predict(X_test)

    # Calculate metrics
    r2 = r2_score(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, y_pred)
    mape = mean_absolute_percentage_error(y_test, y_pred) * 100

    # Print results
    print(f"\n{model_name}:")
    print(f"R²: {r2:.4f}")
    print(f"MSE: {mse:.4f}")
    print(f"RMSE: {rmse:.4f}")
    print(f"MAE: {mae:.4f}")
    print(f"MAPE: {mape: .4f}%")

    results.append([
        model_name,
        round(r2, 4),
        round(mse, 4),
        round(rmse, 4),
        round(mae, 4),
        round(mape, 4)
    ])
```

Create Dataframe

```
results_df = pd.DataFrame(results, columns=["Model", "R²", "MSE", "RMSE", "MAE", "MAPE (%)"])
```

Display table

```
print(results_df)
```

Display as Markdown

```
print(results_df.to_markdown(index=False))
```

MODEL PIPELINE AND TRAINING

```
numeric_features = ["Page Views", "Session Duration", "Bounce Rate", "Time on Page", "Previous Visits"]
categorical_features = ["Traffic Source"]
```

```
preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numeric_features),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
])
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

SELECTING RANDOM FOREST REGRESSOR MODEL

```
print("\n Training Random Forest...")

rf_pipeline = Pipeline([
    ("preprocessing", preprocessor),
    ("regressor", RandomForestRegressor(random_state=42))
])

rf_pipeline.fit(X_train, y_train)
rf_pred = rf_pipeline.predict(X_test)

print("\n Trained Random Forest...")
```

MODEL EVALUATION

Proportion of variance explained by the model (1.0 is perfect) (R2 Score)

```
r2 = r2_score(y_test, rf_pred)
print(f"R2: {r2:.2f}")
```

Average squared difference between predicted and actual values (Mean Squared Error)

```
mse = mean_squared_error(y_test, rf_pred)
print(f"MSE: {mse}")
```

The standard deviation of prediction errors, providing an interpretable error value in the same unit as the target variable (Root Mean Squared Error (RMSE))

```
rmse = np.sqrt(mse)
print(f"RMSE: {rmse}")
```

The average squared difference between predicted and actual values (Root Mean Absolute Error)

```
mae = mean_absolute_error(y_test, rf_pred)
print(f"MAE: {mae}")
```

Compute Mean Absolute Percentage Error (MAPE)

Measures average prediction error in percentage terms

```
mape = np.mean(np.abs((y_test - rf_pred) / y_test)) * 100
print(f"MAPE: {mape:.2f}%")
```

```
regressor = rf_pipeline.named_steps["regressor"]
if hasattr(regressor, "feature_importances_"):

    ohe = rf_pipeline.named_steps["preprocessing"].named_transformers_["cat"]
    onehot_labels = ohe.get_feature_names_out(categorical_features)
    feature_names = numeric_features + list(onehot_labels)

    importances = regressor.feature_importances_
    sorted_idx = np.argsort(importances)[::-1]
```

Feature Importance - Plot

```
plt.figure(figsize=(10, 6))

sns.barplot(x=importances[sorted_idx], y=np.array(feature_names)[sorted_idx])
plt.title("Random Forest Regressor - Feature Importance")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```

Cross Validation Score

```
scores = cross_val_score(rf_pipeline, X, y, cv=5, scoring='r2')
print(f"Cross-validated R² scores: {scores}")
print(f"Average R²: {scores.mean():.3f}")
```

Average Cross Validation Scores

```
print(f"Random Forest Average CV R²: {scores.mean():.3f}")
```

Adjusted R^2 penalizes excessive features to prevent overfitting

```
n = len(y_test)
```

```
p = X_test.shape[1]

adjusted_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)

print(f"Adjusted R²: {adjusted_r2:.4f}")
```

Residual Analysis - Plot Residuals

Helps visualize errors to detect overfitting or underfitting

```
residuals = y_test - rf_pred

plt.figure(figsize=(6, 4))
plt.scatter(y_pred, residuals, color='blue', alpha=0.5)
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel("Predicted Values")
plt.ylabel("Residuals")
plt.title("RF Regressor - Residual Distribution")
plt.show()
```

Residual Normality Test (Shapiro-Wilk Test)

Ensures errors are normally distributed (Ideal for linear regression models)

```
stat, p_value = stats.shapiro(residuals)
print(f"P-value: {p_value:.5f} (Should be > 0.05 for normal distribution)")
```

```
df.dtypes
```

Actual vs Predicted Plot - This plot compares predicted values with actual values

```
plt.figure(figsize=(6, 4))

sns.scatterplot(x=y_test, y=rf_pred, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.title("RF Regressor - Actual vs. Predicted")
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.tight_layout()
plt.show()
```

Quantile Error Metrics - for skewed data, use quantile loss or metrics like median absolute error

```
print(f"Median Absolute Error: {median_absolute_error(y_test, rf_pred):.3f}")
```

Prediction Error Distribution - shows distribution of residuals (errors)

```
errors = y_test - rf_pred
sns.histplot(errors, bins=30, kde=True)
plt.title("RF Regressor - Distribution of Prediction Errors")
plt.xlabel("Error")
plt.show()
```

Compute learning curve data

```
train_sizes, train_scores, test_scores = learning_curve(
    rf_pipeline, X, y, cv=5, scoring='r2',
    train_sizes=np.linspace(0.1, 1.0, 5), random_state=42
)
```

Compute mean and standard deviation

```
train_scores_mean = train_scores.mean(axis=1)
test_scores_mean = test_scores.mean(axis=1)
train_scores_std = train_scores.std(axis=1)
test_scores_std = test_scores.std(axis=1)
```

Learning Curve - Visualizes model performance as training size increases (helps detect overfitting/underfitting)

Plotting the learning curve

```
plt.figure(figsize=(8, 6))
plt.plot(train_sizes, train_scores_mean, 'o-', color="blue", label="Training score")
plt.plot(train_sizes, test_scores_mean, 'o-', color="green", label="Cross-validation score")
```

Shaded std deviation

```
plt.fill_between(
    train_sizes, train_scores_mean - train_scores_std,
    train_scores_mean + train_scores_std, alpha=0.1, color="blue"
)

plt.fill_between(
    train_sizes, test_scores_mean - test_scores_std,
    test_scores_mean + test_scores_std, alpha=0.1, color="green"
)
```

Labels and legend

```
plt.title("📈 Learning Curve - Random Forest")
plt.xlabel("Training Set Size")
plt.ylabel("R² Score")
plt.legend(loc="best")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
# Learning Curve - Visualizes model performance as training size increases (helps detect overfitting/underfitting)


plt.figure(figsize=(8, 6))
```